

Research Proposal

Risk-Asymmetric Selective Prediction for Commercial Aircraft Engine Remaining Useful Life (RUL) Estimation

Team Name: Dragonstone

Course: Advanced Artificial Intelligence

Contents

Background and Introduction to the Area.....	1
Literature Review.....	2
1. Point Estimation Foundations.....	2
2. Bayesian Time-Series Uncertainty.....	3
3. Deep Ensembles and the negative-loglikelihood.....	3
4. Distribution-Free Statistical Guarantees.....	3
5. Risk-Aware Conformalization.....	4
Problem Statement.....	4
Method.....	5
Implemented methodology.....	5
Implementation details.....	5
Key training configuration (selected baseline run).....	5
Experiments and Data.....	6
Dataset used.....	6
Experimental protocol.....	6
Commands used.....	6
Results.....	7
Metrics definition.....	7
Experimental results.....	7
Analysis.....	8
Conclusion.....	8
Appendix A: Reproducibility Artifacts.....	8

Background and Introduction to the Area

Prognostics and Health Management (PHM) has emerged as a fundamental pillar of modern industrial asset management, especially in safety-critical domains such as aerospace engineering. The primary objective of asset prognostics is to accurately estimate the Remaining Useful Life (RUL). The RUL is defined as the time or number of flight cycles left before the equipment component can no longer operate at or above a safe operational threshold.

Historically, maintenance in aircraft engines has relied on hard, schedule-based interventions (e.g., servicing an engine every 500 cycles) or reactive maintenance (fixing a component after failure). Schedule-based frameworks do not permit catastrophic events to occur, but they are economically inefficient because parts with useful service life are often discarded.

With the advent of high-frequency Internet of Things (IoT) sensor arrays on modern turbofan engines, the industry has quickly shifted to Predictive Maintenance (PdM). Modern machine learning (ML) architectures, including Deep Convolutional Networks, Long Short-Term Memory (LSTM) networks and Transformers are trained on multivariate time-series sensor streams (e.g., temperature, pressure, fan speeds) to directly map degradation profiles to a deterministic RUL point estimate.

But the main problem in deploying such models in live aviation operations is this: typical deep learning systems rely on a closed-world assumption and produce single-point predictions without any measure of confidence. Unfortunately, point predictions without formal Uncertainty Quantification (UQ) are operationally non-viable in an environment where over-optimistic prediction can result in inflight engine failure and under-optimistic prediction results in millions of dollars of premature maintenance costs.

The paper treats the evolution of RUL estimation from simple regression to a reliable self-aware Selective Prediction Framework. The system is able to identify its own operational boundaries by providing distribution-free uncertainty bounds for deep learning architectures and automatically ceding highly uncertain predictions to human experts.

Literature Review

Data-driven RUL estimation with the benchmark NASA C-MAPSS dataset has been widely studied in the academic literature from statistical modeling to very complex deep learning. Recently, a distinct shift towards quantification of the reliability of these models has emerged. Five key papers outline the development and existing gaps within this domain:

1. Point Estimation Foundations

Babu, G. S., Zhao, P., & Li, X.-L. (2016). "Deep Convolutional Neural Network Based Regression Approach for Remaining Useful Life Estimation." International Conference on Neural Information Processing.

Contribution: This seminal paper was one of the first to move away from hand-crafted statistical features by directly applying Deep Convolutional Neural Networks (CNN) on multivariate time-series data from the C-MAPSS dataset. The authors demonstrated that an auto sliding-window feature extractor can achieve better performance than traditional Support Vector Regression (SVR) and Multi-Layer Perceptron (MLPs).

Limitation: Deterministic architecture only. Predictions are treated the same, with no way to indicate that test data is outside the training data distribution.

2. Bayesian Time-Series Uncertainty

Chen, S., He, J., Wen, P., et al. (2023). Convolutional Bayesian Long Short-Term Memory Neural Network for Remaining Useful Life Prognostics and Uncertainty Quantification of Aircraft Engines IEEE Conference on Prognostics & Health Management.

Contribution: The authors have developed a hybrid Convolutional Bayesian LSTM (CB-LSTM) networks. The model captures Epistemic Uncertainty by placing a prior distribution over the neural network weights and doing variational inference. (model uncertainty caused by limited training data) but also providing accurate point predictions.

Limitation: Bayesian Neural Networks (BNNs) require computing a complex posterior distribution that is computationally expensive and slow for real-time edge streaming. In addition, the resulting prediction intervals are very sensitive to the choice of an appropriate prior distribution and to model misspecification.

3. Deep Ensembles and the negative-loglikelihood

Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). "Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles." *NeurIPS*. (Applied to C-MAPSS via OpenReview, 2024)

Contribution: This framework overcame the computational challenges of BNNs by training an ensemble of independent deterministic LSTMs. Importantly, we modified the models to output a mean (μ) and variance (σ^2) for each step and trained them to minimize a Negative Log-Likelihood (NLL) loss function to learn Aleatoric Uncertainty. (inherent data noise)

Limitation: Deep ensembles are very scalable, but do not have any theoretical guarantees. However, if the entire ensemble is uniformly overconfident on a highly anomalous out-of-distribution engine profile, the aggregated uncertainty metric can still fail to flag the danger.

4. Distribution-Free Statistical Guarantees

Javanmardi, H., & Hüllermeier, E. (2023). "Conformal Prediction Intervals for Remaining Useful Lifetime Estimation." *International Journal of Prognostics and Health Management*.

Contribution: This paper proposed the Conformal Prediction (CP) framework for the RUL problem. CP is a post-processing wrapper that can be applied to any pre-trained model (e.g. Gradient Boosting, CNNs). With the mild assumption of data exchangeability, it provides valid prediction intervals that ensure that the true RUL will lie in the bounded range at a user-specified confidence level (e.g., 90%).

Limitation: The generated intervals are symmetric about the prediction point, so that early maintenance penalties and late maintenance catastrophes are given equal mathematical weight.

5. Risk-Aware Conformalization

Robinson, C. (2026). "Remaining Useful Life Estimation for Aircraft Engines with Risk-Aware Prediction Intervals via Conformalized Quantile Regression." *International Journal of Prognostics and Health Management*.

Contribution: The symmetry problem has been addressed in a recently published paper with an asymmetric formulation of Conformalized Quantile Regression (CQR). It

intentionally skews the miscoverage error bound towards safety, in an effort to have the model minimize the risks of late maintenance interventions.

Limitation: The intervals are risk-aware, but the model does not have an automated Selective Prediction mechanism. It does not actively address the operational problem of dynamically routing high-risk/high-uncertainty engine assets away from automation to human-in-the-loop review systems.

Method

Implemented methodology

The implemented pipeline follows four stages:

- **Sliding-window feature construction** from C-MAPSS trajectories.
- **TCN quantile regression** to predict lower/median/upper RUL estimates.
- **Asymmetric conformal calibration** to enforce uncertainty control while penalizing optimistic (unsafe) misses more strongly.
- **Selective routing** to accept only predictions with interval width below a threshold; otherwise abstain/escalate.

Implementation details

- Framework: Python 3.12 + PyTorch (CUDA)
- GPU setup: Dual NVIDIA RTX A2000 12GB
- Training script: `train.py`
- Main package: `src/rul_selective/`

Key training configuration (selected baseline run)

- Epochs: 30
- Batch size: 128
- Learning rate: 1e-3
- Window size: 30
- Calibration fraction: 0.20
- Alpha: 0.10
- Overestimate penalty: 3.0
- Max interval width: 35.0
- Seed: 7

Experiments and Data

Dataset used

- Dataset: NASA C-MAPSS FD001 (text format)
- Files:
 - data/cmapss/train_FD001.txt
 - data/cmapss/test_FD001.txt
 - data/cmapss/RUL_FD001.txt
- File sizes by row count:
 - Train rows: 20,631
 - Test rows: 13,096
 - RUL labels: 100

Experimental protocol

The training engines were split unit-wise into the following:

- 80 engines for model training
- 20 engines for conformal calibration

Evaluation was then performed on the FD001 test set using the provided RUL labels.

Commands used

```
PROJECT_ROOT=/path/to/risk-asymmetric-rul
cd "$PROJECT_ROOT"
source "$PROJECT_ROOT/venv/bin/activate"

python train.py \
  --train-file "$PROJECT_ROOT/data/cmapss/train_FD001.txt" \
  --test-file "$PROJECT_ROOT/data/cmapss/test_FD001.txt" \
  --rul-file "$PROJECT_ROOT/data/cmapss/RUL_FD001.txt" \
  --output-dir "$PROJECT_ROOT/runs/fd001_provided_eval"
```

Secondary tuning experiment:

```
python train.py \
  --train-file "$PROJECT_ROOT/data/cmapss/train_FD001.txt" \
  --test-file "$PROJECT_ROOT/data/cmapss/test_FD001.txt" \
  --rul-file "$PROJECT_ROOT/data/cmapss/RUL_FD001.txt" \
  --output-dir "$PROJECT_ROOT/runs/fd001_tuned" \
```

```
--alpha 0.05 \
--calibration-fraction 0.30 \
--overestimate-penalty 4.0 \
--max-interval-width 30
```

Results

Metrics definition

- **Coverage:** percentage of true RUL values in predicted intervals.
- **Unsafe lower miss rate:** fraction of cases where the true RUL is below the predicted lower bound (optimistic/risky misses).
- **Abstention rate:** percentage of samples raised for high uncertainty.
- **Median MAE:** absolute error of the median (point) prediction of RUL.

Experimental results

Run	Coverage	Unsafe lower miss	Upper miss	Mean interval width	Abstention	Median MAE	Accepted median MAE
Baseline (fd001_provided_eval)	0.63	0.17	0.2	24.61	0.08	12.32	11.63
Tuned (fd001_tuned)	0.58	0.18	0.24	24.51	0.41	12.45	11.4

Analysis

The baseline run is the better overall operating point for this project:

- Higher coverage (0.63 vs 0.58)
- Lower unsafe lower misses (0.17 vs 0.18)
- Much lower abstention (0.08 vs 0.41)
- Slightly better median MAE (12.32 vs 12.45)

Although the tuned run slightly improved accepted-only MAE, it rejected too many cases and degraded reliability metrics.

Conclusion

The proposed risk-asymmetric selective RUL framework was successfully applied to C-MAPSS FD001. The model is able to provide uncertainty-aware RUL estimates while keeping a practical abstention rate.

For this implementation, the baseline configuration provides the best tradeoff between coverage, safety (unsafe miss control) and automation rate. Future work can improve calibration quality and robustness with richer feature engineering, alternative sequence architectures, and multi-condition datasets (e.g., FD004/N-MAPSS conversions).

Appendix A: Reproducibility Artifacts

- Model checkpoint: `runs/fd001_provided_eval/model.pt`
- Run metadata: `runs/fd001_provided_eval/metadata.json`
- Test metrics: `runs/fd001_provided_eval/test_metrics.json`

Appendix B: Command Execution Outputs

Baseline run (fd001_provided_eval)

Command:


```
python    train.py    --train-file    data/cmapss/train_FD001.txt
--test-file    data/cmapss/test_FD001.txt    --rul-file
data/cmapss/RUL_FD001.txt --output-dir runs/fd001_provided_eval
```

Console output:

```
epoch=001 train_loss=14.72066
epoch=002 train_loss=8.78889
epoch=003 train_loss=8.32907
epoch=004 train_loss=7.47940
epoch=005 train_loss=7.17975
epoch=006 train_loss=6.69193
epoch=007 train_loss=6.45197
epoch=008 train_loss=6.24282
epoch=009 train_loss=6.06481
epoch=010 train_loss=5.72721
epoch=011 train_loss=5.56796
epoch=012 train_loss=5.46295
epoch=013 train_loss=5.35695
epoch=014 train_loss=5.14336
epoch=015 train_loss=5.09941
epoch=016 train_loss=5.17102
epoch=017 train_loss=4.94312
epoch=018 train_loss=5.02103
```

```
epoch=019 train_loss=4.91615
epoch=020 train_loss=4.79785
epoch=021 train_loss=4.62371
epoch=022 train_loss=4.59665
epoch=023 train_loss=4.59065
epoch=024 train_loss=4.55314
epoch=025 train_loss=4.47824
epoch=026 train_loss=4.47921
epoch=027 train_loss=4.40204
epoch=028 train_loss=4.43543
epoch=029 train_loss=4.25760
epoch=030 train_loss=4.27919

{
  "device": "cuda",
  "gpu_count": 2,
  "train_engines": 80,
  "calibration_engines": 20,
  "calibration_lower_correction": 0.6165342926979065,
  "calibration_upper_correction": 0.0
}

{
  "coverage": 0.63,
  "unsafe_lower_miss_rate": 0.17,
```

```
"upper_miss_rate": 0.2,  
"mean_interval_width": 24.605016708374023,  
"abstention_rate": 0.08,  
"median_mae": 12.321646690368652,  
"accepted_median_mae": 11.630104064941406  
}
```

C.2 Tuned run (fd001_tuned)

Command:

```
python      train.py      --train-file      data/cmapss/train_FD001.txt      --test-file  
data/cmapss/test_FD001.txt      --rul-file      data/cmapss/RUL_FD001.txt      --output-dir  
runs/fd001_tuned --alpha 0.05 --calibration-fraction 0.30 --overestimate-penalty 4.0  
--max-interval-width 30
```

Console output:

```
epoch=001 train_loss=16.31389  
epoch=002 train_loss=9.96593  
epoch=003 train_loss=9.05868  
epoch=004 train_loss=8.56957  
epoch=005 train_loss=7.89049  
epoch=006 train_loss=7.63132  
epoch=007 train_loss=7.49858  
epoch=008 train_loss=7.04528  
epoch=009 train_loss=6.67240  
epoch=010 train_loss=6.49427
```

```
epoch=011 train_loss=6.32329
epoch=012 train_loss=6.38879
epoch=013 train_loss=5.91980
epoch=014 train_loss=6.08195
epoch=015 train_loss=5.85022
epoch=016 train_loss=6.02561
epoch=017 train_loss=5.43658
epoch=018 train_loss=5.58992
epoch=019 train_loss=5.54078
epoch=020 train_loss=5.35136
epoch=021 train_loss=5.37084
epoch=022 train_loss=5.04142
epoch=023 train_loss=5.28131
epoch=024 train_loss=5.01758
epoch=025 train_loss=5.19010
epoch=026 train_loss=5.10355
epoch=027 train_loss=4.91847
epoch=028 train_loss=4.90055
epoch=029 train_loss=4.82018
epoch=030 train_loss=4.95324
{
  "device": "cuda",
  "gpu_count": 2,
```

```
"train_engines": 70,  
"calibration_engines": 30,  
"calibration_lower_correction": 0.08663015067577362,  
"calibration_upper_correction": 0.0  
}  
  
{  
  "coverage": 0.58,  
  "unsafe_lower_miss_rate": 0.18,  
  "upper_miss_rate": 0.24,  
  "mean_interval_width": 24.509077072143555,  
  "abstention_rate": 0.41,  
  "median_mae": 12.448171615600586,  
  "accepted_median_mae": 11.395181655883789  
}
```